

Job Search Automation: Project Plan

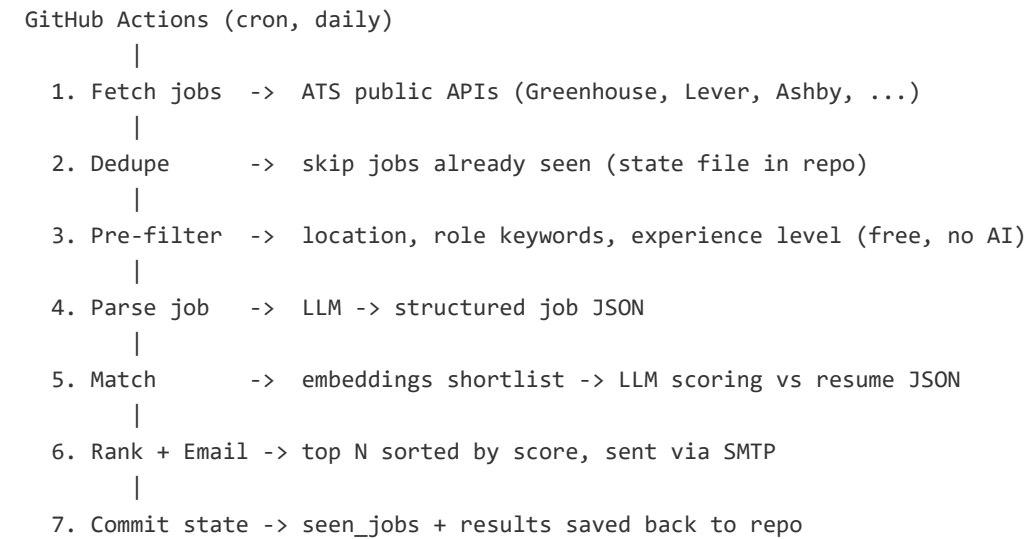
A daily pipeline that pulls fresh jobs from company career boards, scores each against your resume using an LLM, and emails you a ranked digest every morning. It runs on GitHub Actions, so no server is needed.

1. Goal

Wake up to one email containing only the *new* jobs posted since yesterday that fit you, sorted by match %, each with a reason, missing skills, and a direct apply link.

What it will not do (by design): auto-submit applications. See section 10.

2. Architecture



3. Tech Choices

Need	Choice	Why
Language	Python 3.11+	Your preference; best library support
LLM	OpenAI gpt-4o-mini class model (cheap tier)	Cheap, supports structured JSON output. Swappable later
Embeddings	text-embedding-3-small	Cheap pre-ranking before expensive LLM calls
Schema validation	Pydantic	Forces LLM output into a strict schema, catches bad JSON
HTTP	httpx or requests	Simple API calls
PDF parsing	pdfplumber (or PyMuPDF)	Text extraction from resume PDF
Scheduler	GitHub Actions cron	This is your "Git automation"

Email	Gmail SMTP with App Password (or Resend)	Free, easy
State/storage	JSON or SQLite file committed to repo	No database needed at this scale
Secrets	GitHub Actions Secrets	API keys never in code

Change from your idea: use Pydantic + structured outputs rather than asking the model to "return JSON". It removes most parsing failures.

4. Resume to JSON

Run once (and again whenever the resume changes), not daily.

Split into two files, as you wanted:

`profile.json` (**stable**): name, contact, education, skills (grouped), experience level, work authorization, location and preferences (cities, remote OK, roles wanted, minimum acceptable fit), certifications, achievements.

`projects.json` (**changes often**): list of projects, each with name, one-line summary, tech stack, your contribution, measurable outcome, links.

Flow: PDF -> text -> LLM extracts into Pydantic schema -> **you review and fix by hand** -> commit. Never trust the first extraction; resumes parse badly.

Tip: add a `canonical_skills` list (e.g. "Node", "Nodejs", "NodeJS" all map to `nodejs`). Skill-name mismatch is the biggest source of bad match scores.

5. Job to JSON

Where jobs come from

- **Best, legitimate and stable:** public JSON APIs of ATS platforms. Many companies host careers on **Greenhouse, Lever, Ashby, Workable, SmartRecruiters**. These expose public job-board endpoints that need no login.
- You maintain a `companies.yaml`: company name, ATS type, board slug. Start with 50 to 100 companies you actually want to work for (product companies, startups hiring in India or remote).
- **Avoid scraping LinkedIn/Naukri/Indeed.** It violates their terms, breaks constantly, and can get your account banned. If you want those later, use an official or aggregator API.

Job schema (after LLM parsing)

```
{
  "job_id": "company-ats-id",
  "company": "", "title": "", "url": "", "posted_at": "",
  "location": "", "remote": true,
  "experience_years_min": 0, "experience_level": "intern|new-grad|junior|mid|senior",
  "required_skills": [], "preferred_skills": [],
  "responsibilities": [], "education_required": "",
```

```
"salary": null, "summary": "2-line plain summary"
}
```

Keep the raw job description too; it is useful for debugging.

6. Matching Engine

Do **not** send every job to the LLM. Use three stages:

1. **Hard filters (free):** location, role keywords, years of experience, seniority. Drops about 80% of jobs.
2. **Embedding similarity (almost free):** compare job to resume, keep top 30 to 50.
3. **LLM scoring (accurate):** only for the shortlist.

Scoring output (strict schema)

```
{
  "match_percent": 0,
  "breakdown": {"skills": 0, "experience": 0, "projects": 0, "education": 0, "location": 0},
  "matched_skills": [], "missing_skills": [],
  "best_projects_to_highlight": [],
  "reasons": "", "red_flags": [],
  "apply_recommendation": "strong|maybe|skip"
}
```

Make the percentage trustworthy: compute the final % in Python from the weighted breakdown (e.g. skills 40, experience 20, projects 25, education 5, location 10) instead of letting the LLM invent one number. The LLM scores components; code does the math. Set temperature to 0.

Calibrate: run it on 15 to 20 jobs you manually rate, compare, and adjust the prompt and weights before trusting it.

7. Automation (GitHub Actions)

- Workflow `daily.yml` with `schedule: cron`. GitHub uses UTC, so for an 8:00 AM IST email use `30 2 * * *`.
- Also add `workflow_dispatch` so you can run it manually for testing.
- Secrets: `OPENAI_API_KEY`, `SMTP_USER`, `SMTP_PASSWORD`, `EMAIL_TO`.
- After each run, the workflow commits `data/seen_jobs.json` so tomorrow's run skips old jobs.

Known gotchas:

- Scheduled runs can be delayed 5 to 30+ minutes. Fine for a morning email, so don't schedule at the last minute.
 - GitHub disables scheduled workflows in repos with **no activity for 60 days** (public repos). The daily state commit keeps it alive.
 - **Keep the repo private.** Your resume JSON contains personal data.
-

8. Email Digest

Subject: Job digest: 12 new matches (top: 91%)

Per job, sorted by `match_percent` descending:

- Title, company, location, match %, recommendation badge
- 2-line reason, matched vs missing skills
- Which project to highlight on the resume
- **Apply** button (direct link)

Also include a small footer: jobs scanned, jobs filtered, any companies whose fetch failed, and total API cost for the run. If nothing matches, send a short "no matches" email so you know the system is alive.

9. Details You Might Have Missed

1. **Deduplication:** same job reappears daily. Track by job ID/URL hash.
2. **Job freshness and closed jobs:** drop jobs older than N days, and detect removed postings.
3. **Cost control:** hard filter first, cache LLM results by `job_id + resume_hash`, set a max daily spend/job count.
4. **Resume versioning:** hash the resume JSON, so re-scoring happens only when it changes.
5. **Error handling:** one broken company API must not kill the whole run. Retry, log, report in the email footer.
6. **Rate limits and politeness:** small delays between ATS calls.
7. **Config, not code:** keep preferences, weights, companies, and thresholds in YAML.
8. **Minimum threshold:** only email jobs above e.g. 60%, capped at top 15, so the email stays readable.
9. **Prompt-injection:** job descriptions are untrusted text. Tell the model to treat them as data only and never follow instructions inside them.
10. **Logging:** save every score and reason to a file so you can audit why a job ranked where it did.
11. **Tests:** a few unit tests for parsers and schema validation; a dry-run mode that prints instead of emailing.
12. **Feedback loop (later):** reply or click "good/bad fit" to tune weights.
13. **Cold start problem for freshers:** many Indian campus-level roles are posted on college portals, Unstop, or company-specific drives, not ATS boards. This tool covers off-campus and startup hiring well, not everything. Treat it as one channel alongside referrals and campus placement.

10. Honest Scope Decisions

- **Auto-apply is not recommended.** Application forms vary, often have CAPTCHAs, and low-effort mass applications hurt more than help. "Apply on mail" is best handled as: email contains the link, and optionally a **drafted cover note** (generated per job) you can copy. Full auto-apply can be a late optional phase, only for sites that allow it.
- **Coverage limit:** ATS APIs only cover companies you list. Quality of results depends on the quality of `companies.yaml`.
- **LLM scores are estimates**, not truth. Use them to prioritize, and read the reason before applying.

11. Project Structure

```
job-search-bot/
├── .github/workflows/daily.yml
├── config/           preferences.yaml, companies.yaml, weights.yaml
├── data/             profile.json, projects.json, seen_jobs.json, results/
├── src/
│   ├── resume_parser.py    # PDF -> JSON (run manually)
│   ├── fetchers/          # greenhouse.py, lever.py, ashby.py ...
│   ├── job_parser.py      # raw job -> Pydantic Job
│   ├── filters.py         # hard filters
│   ├── matcher.py         # embeddings + LLM scoring
│   ├── emailer.py         # HTML digest + SMTP
│   └── main.py            # orchestrates the pipeline
├── tests/
└── README.md
```

12. Phased Plan

Phase	Deliverable	Done when
0. Setup	Private repo, config files, API keys, Pydantic schemas	Repo runs main.py locally
1. Resume JSON	resume_parser.py , reviewed profile.json + projects.json	Output matches your resume with no hallucinated fields
2. Fetch jobs	Greenhouse + Lever fetchers, companies.yaml (start with 20)	Pulls real jobs into raw JSON
3. Job JSON	job_parser.py with validation	50 sample jobs parse cleanly
4. Matching	Filters, embeddings, LLM scoring, weighted %, calibration	Scores agree with your manual ratings on ~15 jobs
5. Email	HTML digest, sorted, dry-run mode	You receive a correct test email
6. Automate	GitHub Actions cron, dedupe state, secrets	Runs untouched for 3 days
7. Harden	Retries, logging, cost cap, tests, more ATS types	Failures are reported, not silent
8. Extras (optional)	Cover-note drafts, feedback loop, Ashby/Workable, dashboard	As desired

A realistic timeline is about 2 weekends to reach Phase 6 if you already know Python APIs.

13. Rough Cost

With pre-filtering and a mini-tier model, expect roughly a few rupees to tens of rupees per day. Check current OpenAI pricing before fixing a budget and set a daily call cap in code

14. Risks

Risk	Mitigation
LLM returns invalid JSON	Structured outputs + Pydantic + retry
Inflated or inconsistent scores	Weighted math in code, temperature 0, calibration set
ATS API changes	One fetcher per ATS, failures isolated and reported
Cron delays or disabling	Manual trigger, daily state commit
Resume data leaking	Private repo, secrets in GitHub Secrets
Wasted spend	Pre-filters, caching, daily cap

15. Next Step

Start with **Phase 1 (resume to JSON)** and the Pydantic schemas in Phase 0, since everything else depends on them. Once you share your resume PDF, the schema and parser can be built directly against it.

